Week 5 Lab 2: Document Data Modelling and NoSQL Persistence

Purpose

In Lab 1, you mapped a domain model to a relational model and implemented persistence using Entity Framework Core and SQLite.

In this lab, you will introduce **Research Activity Log** as a new business capability. You will compare relational and document representations before implementing document persistence.

The purpose is not simply to use a NoSQL database. The purpose is to understand how the structure, variability, relationships, and access patterns of data influence the persistence model.

Learning Outcomes

By completing the core lab, you should be able to:

- Compare relational and document representations of the same domain concept.
- Design a document model around required application access patterns.
- Implement a small document-persistence boundary in an ASP.NET Core MVC module.
- Display persisted document data through a controller, service, repository, and Razor view.

Use-Case Descriptions

The Research Activity Log use-case descriptions will be provided in a separate file.

The core lab uses the use case for viewing a student's research activities. A record-activity use case may be completed as an extension.

Read the supplied use case before implementing the persistence classes. Use it to identify:

- The information that must be stored.
- The activity variants the model must support.
- The required query and ordering rule.
- The fields displayed in the view.
- The supplied alternative flow.

Persistence Scope

Lab 2 introduces a new **Research Activity Log** module.

The module uses document persistence. Existing Students-module relational persistence and Academic Supervisor persistence should continue to work unchanged.

Do not move existing relational tables into the document database.

Files and Project Required

Before starting, make sure you have:

- Your completed Week 5 Lab 1 project.
- The separately supplied Research Activity Log use-case descriptions.
- The Week 5 Lab 1 domain models and ERDs.
- Your updated implementation class diagram, if Extension 1 was completed.
- Any tests and design notes retained from Lab 1.

As a last resort, you can use the starter project for this week.

Part 1: Preserve the Lab 1 Baseline

Task 1: Check the Existing Application

Before adding document persistence:

1. Build and run the Lab 1 project.
2. Confirm that Student and Enrolment data is loaded from SQLite.
3. Confirm that seeded enrolments appear in the HTML view.
4. Confirm that Academic Supervisor Management still works.
5. Save or commit a working checkpoint.

```
git status
git add .
git commit -m "Preserve working Week 5 relational baseline"
```

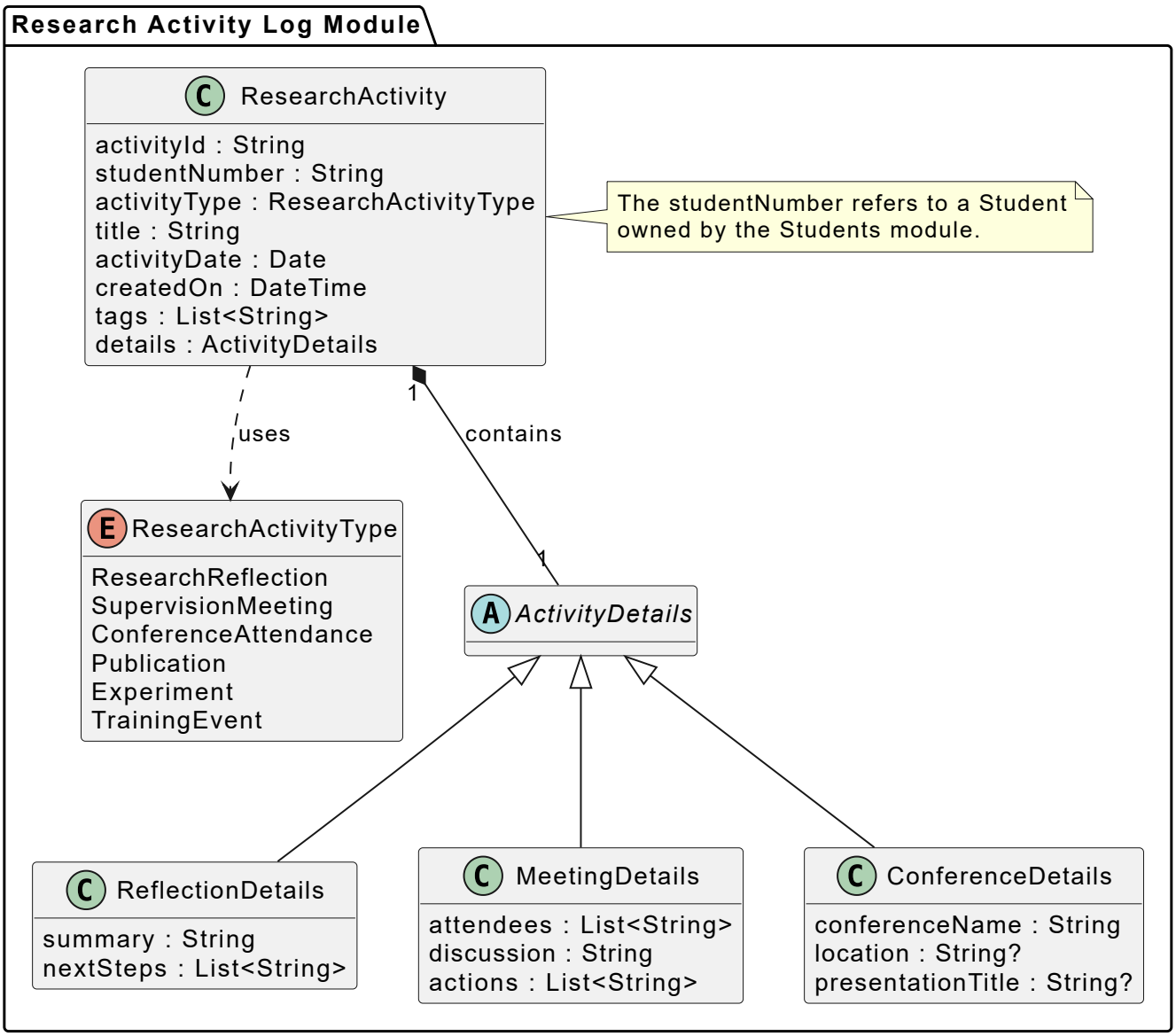
Part 2: Understand the Research Activity Domain

Initial Research Activity Domain Model

A `ResearchActivity` represents a dated activity recorded as part of a student's research journey.

All activities share common information, but different activity types may require different details.

Week 5 Research Activity Domain Model



Task 2: Analyse the Domain Model

Identify:

- The fields shared by every research activity.
- The details that vary by activity type.
- The information that belongs inside the activity.
- The identifier that refers to the Student owned by another module.
- The activity types needed by the supplied use case.

Record whether every type in the diagram is required for your core implementation. For the core lab, you may implement only the activity types represented by the supplied seed data.

Part 3: Compare Relational and Document Models

Task 3: Attempt a Relational Representation

Before using a document database, sketch how the Research Activity Log could be represented relationally.

Consider at least these two options.

Option A: One Wide Table

```
ResearchActivity
    activity_id
    student_number
    activity_type
    title
    activity_date
    created_on
    summary
    next_steps
    attendees
    discussion
    actions
    conference_name
    location
    presentation_title
```

Option B: Shared Table with Type-Specific Tables

```
ResearchActivity
    1 -> 0..1 ReflectionDetails
    1 -> 0..1 MeetingDetails
    1 -> 0..1 ConferenceDetails
```

For each option, consider:

- Nullable columns.
- Additional tables and joins.
- Constraints that ensure the correct details are used for each type.
- Adding a new activity type.
- Retrieving all activities for one student.

Do not implement the relational model. Keep the sketch for comparison with the document model.

Task 4: Review the Initial Document Model

The core document model stores information that is read together in one activity document.

```
{
  "_id": "activity-001",
  "studentNumber": "S100001",
  "activityType": "ResearchReflection",
  "title": "Review of initial literature search",
  "activityDate": "2026-02-12",
  "createdOn": "2026-02-12T15:30:00Z",
```

```
  "tags": ["literature", "planning"],
  "details": {
    "summary": "Reviewed the initial search results and refined the research
question.",
    "nextSteps": [
      "Repeat the search using the revised terms",
      "Discuss the inclusion criteria at the next meeting"
    ]
  }
}
```

A second activity may have a different `details` shape:

```
{
  "_id": "activity-002",
  "studentNumber": "S100001",
  "activityType": "SupervisionMeeting",
  "title": "Monthly supervision meeting",
  "activityDate": "2026-03-04",
  "createdOn": "2026-03-04T12:00:00Z",
  "tags": ["supervision", "methods"],
  "details": {
    "attendees": ["S100001", "SUP001"],
    "discussion": "Reviewed the proposed data-collection method.",
    "actions": [
      "Revise the participant information sheet",
      "Prepare the ethics application"
    ]
  }
}
```

Explain:

- Why the type-specific details are embedded.
- Why `studentNumber` is stored as a reference rather than embedding the complete Student.
- Which fields should be indexed.
- How the model supports retrieving all activities for one student.
- How a new activity type could be introduced.

Part 4: Implement the Document Model

Class and File Locations

Use these locations:

```
PgrStudentManagement.Web/
└── Modules/
    └── ResearchActivities/
```

```
        ├── Controllers/
        │   └── ResearchActivityController.cs
        ├── Models/
        │   ├── ResearchActivity.cs
        │   └── ResearchActivityType.cs
        ├── Data/
        │   ├── ResearchActivityRepository.cs
        │   └── ResearchActivityDataSeeder.cs
        └── Services/
            └── ResearchActivityService.cs

Views/
└── ResearchActivities/
    └── Index.cshtml
```

If your project uses different internal folder names, apply one convention consistently and keep the new classes inside the Research Activity Log module.

Task 5: Add the MongoDB Driver and Configuration

Add the MongoDB .NET driver using a version compatible with the project:

```
dotnet add package MongoDB.Driver
```

Add the connection settings to `appsettings.json`:

```
{
  "ResearchActivityDatabase": {
    "ConnectionString": "mongodb://localhost:27017",
    "DatabaseName": "StudentManagement",
    "CollectionName": "ResearchActivities"
  }
}
```

Do not commit real usernames, passwords, or connection secrets.

Your tutor will confirm whether the lab uses a local MongoDB instance, a container, or another provided development environment.

Task 6: Create the Document Classes

Create:

```
Modules/ResearchActivities/Models/ResearchActivity.cs
Modules/ResearchActivities/Models/ResearchActivityType.cs
```

Use a document representation that can store common fields and activity-specific details.

A suitable core implementation is:

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace PgrStudentManagement.Web.Modules.ResearchActivities.Models;

public class ResearchActivity
{
    [BsonId]
    public string Id { get; set; } = string.Empty;

    public string StudentNumber { get; set; } = string.Empty;

    public ResearchActivityType ActivityType { get; set; }

    public string Title { get; set; } = string.Empty;

    public DateTime ActivityDate { get; set; }

    public DateTime CreatedOn { get; set; }

    public List<string> Tags { get; set; } = [];

    public Dictionary<string, object> Details { get; set; } = new();
}
```

```
namespace PgrStudentManagement.Web.Modules.ResearchActivities.Models;

public enum ResearchActivityType
{
    ResearchReflection,
    SupervisionMeeting,
    ConferenceAttendance,
    Publication,
    Experiment,
    TrainingEvent
}
```

The domain model describes activity-specific details conceptually using separate detail types.

The core implementation simplifies this model by storing details in a `Dictionary<string, object>`. Strongly typed detail classes are an extension.

The conceptual model and core implementation are therefore related but not structurally identical.

Task 7: Create and Register the Repository

Create:

```
Modules/ResearchActivities/Data/ResearchActivityRepository.cs
```

Use:

```
using MongoDB.Driver;
using PgrStudentManagement.Web.Modules.ResearchActivities.Models;

namespace PgrStudentManagement.Web.Modules.ResearchActivities.Data;

public class ResearchActivityRepository
{
    private readonly IMongoCollection<ResearchActivity> _activities;

    public ResearchActivityRepository(
        IMongoClient client,
        IConfiguration configuration)
    {
        var databaseName =
configuration["ResearchActivityDatabase:DatabaseName"]!;

        var collectionName =
configuration["ResearchActivityDatabase:CollectionName"]!;

        var database = client.GetDatabase(databaseName);

        _activities = database.GetCollection<ResearchActivity>(collectionName);
    }

    public async Task<IReadOnlyList<ResearchActivity>> GetForStudentAsync(string
studentNumber)
    {
        return await _activities
            .Find(activity =>
                activity.StudentNumber == studentNumber)
            .SortBy(activity => activity.ActivityDate)
            .ToListAsync();
    }

    public Task AddAsync(ResearchActivity activity)
    {
        return _activities.InsertOneAsync(activity);
    }
}
```

Register the MongoDB client and repository in `Program.cs`:

```
builder.Services.AddSingleton<IMongoClient>(
    _ => new MongoClient(
        builder.Configuration[
```

```
            "ResearchActivityDatabase:ConnectionString"]));

builder.Services.AddScoped<ResearchActivityRepository>();
```

The repository owns document-database access. Controllers and views must not use `IMongoCollection<T>` directly.

Task 8: Seed the Research Activities

Create:

```
Modules/ResearchActivities/Data/ResearchActivityDataSeeder.cs
```

Seed at least:

- Two activities for `S100001` with different activity types.
- One activity for `S100002`.
- One Student with no activities for testing the alternative flow.

Use stable test content and no personal or confidential data.

A possible seed method is:

```
public static async Task SeedAsync(ResearchActivityRepository repository)
{
    var existing = await repository.GetForStudentAsync("S100001");

    if (existing.Count > 0)
    {
        return;
    }

    await repository.AddAsync(new ResearchActivity
    {
        Id = "activity-001",
        StudentNumber = "S100001",
        ActivityType = ResearchActivityType.ResearchReflection,
        Title = "Review of initial literature search",
        ActivityDate = new DateTime(2026, 2, 12),
        CreatedOn = DateTime.UtcNow,
        Tags = ["literature", "planning"],
        Details = new Dictionary<string, object>
        {
            ["summary"] = "Reviewed the initial search results.",
            ["nextSteps"] = new[]
            {
                "Repeat the search using revised terms",
                "Discuss the inclusion criteria"
            }
        }
```

```
    });

    await repository.AddAsync(new ResearchActivity
    {
        Id = "activity-002",
        StudentNumber = "S100001",
        ActivityType = ResearchActivityType.SupervisionMeeting,
        Title = "Monthly supervision meeting",
        ActivityDate = new DateTime(2026, 3, 4),
        CreatedOn = DateTime.UtcNow,
        Tags = ["supervision", "methods"],
        Details = new Dictionary<string, object>
        {
            ["discussion"] = "Reviewed the proposed data-collection method.",
            ["actions"] = new[]
            {
                "Revise the participant information sheet",
                "Prepare the ethics application"
            }
        }
    });

    await repository.AddAsync(new ResearchActivity
    {
        Id = "activity-003",
        StudentNumber = "S100002",
        ActivityType = ResearchActivityType.ConferenceAttendance,
        Title = "Postgraduate research conference",
        ActivityDate = new DateTime(2026, 3, 18),
        CreatedOn = DateTime.UtcNow,
        Tags = ["conference", "presentation"],
        Details = new Dictionary<string, object>
        {
            ["conferenceName"] = "Postgraduate Research Conference",
            ["location"] = "Sheffield",
            ["presentationTitle"] = "Initial Findings from the Research Project"
        }
    });

}
```

Run the seeder from a dependency-injection scope during application startup. Do not insert duplicate seed records on every restart.

The Lab 1 relational database must contain at least one known student for whom no Research Activity documents are seeded. Use that student to test the **No Research Activities Found** alternative flow.

This simple seed check assumes that the complete seed dataset is either present or absent. If the database contains a partial seed dataset, delete the development collection and run the seeder again.

Task 9: Add the Service, Controller, and View

Create:

```
Modules/ResearchActivities/Services/ResearchActivityService.cs
Modules/ResearchActivities/Controllers/ResearchActivityController.cs
Views/ResearchActivities/Index.cshtml
```

The intended dependency direction is:

```
ResearchActivityController
    -> ResearchActivityService
        -> ResearchActivityRepository
            -> MongoDB
```

The service should expose an operation such as:

```
public Task<IReadOnlyList<ResearchActivity>> GetActivitiesForStudentAsync(string
studentNumber)
```

The controller should:

- Receive the student number.
- Call the service.
- Return the Research Activities view.
- Display the supplied empty-result alternative when no activities are found.

The Razor view should display at least:

- Activity type
- Title
- Activity date
- Tags
- Selected detail information
    - Research reflection: display `summary`
    - Supervision meeting: display `discussion`
    - Conference attendance: display `conferenceName`

Add a link from the appropriate Student page to the Research Activity Log.

For the core implementation, an empty repository result may produce **No research activities found** without first checking whether the student exists.

The Research Activity Log-to-Students relationship is optional in the core implementation. It becomes an implemented dependency only if Student existence is checked through `IStudentLookup`.

Checking student existence through the Students-module public contract is an extension.

Task 10: Verify the Core Use Case

Run the application and verify through the browser that:

- `S100001` displays activities in chronological order.
- Different activity types can contain different detail fields.
- `S100002` displays its seeded activity.
- A student with no activities displays the supplied alternative response.
- Existing relational Student and Enrolment behaviour still works.
- Existing Academic Supervisor behaviour still works.
- The controller calls the service.
- The service calls the repository.
- The repository is the only application class that accesses the MongoDB collection.

Stop and restart the application. Confirm that the activity documents remain available and are not duplicated by the seeder.

Core Completion Checklist

- [ ] I preserved the working Lab 1 application.
- [ ] I analysed the Research Activity domain model.
- [ ] I compared relational and document representations.
- [ ] I identified embedding and reference decisions.
- [ ] I configured MongoDB document persistence.
- [ ] I created the Research Activity document classes.
- [ ] I created and registered the repository.
- [ ] I seeded multiple activity types.
- [ ] I added the service, controller, and Razor view.
- [ ] I viewed persisted activities through the application.
- [ ] I verified the empty-result alternative.
- [ ] Existing relational and Academic Supervisor behaviour still works.

Extension Tasks

Extension 1: Record a Research Activity

Implement the supplied use case for recording a new Research Activity.

Add a form model and Razor view. Validate common fields and the detail fields required by the selected activity type. Use the service and repository rather than accessing MongoDB from the controller.

Extension 2: View Research Activity Details

Implement `W5-UC05`: View Research Activity Details.

Add a repository operation that retrieves one activity using its unique identifier.

Add the corresponding service operation, controller action, and Razor view.

The details page should display:

- Activity type
- Title

- Activity date
- Creation date and time
- Tags
- All stored activity-specific details

Implement the supplied **Research Activity Not Found** alternative flow.

Extension 3: Use Strongly Typed Detail Classes

Replace `Dictionary<string, object>` with a typed detail hierarchy or another deliberate polymorphic representation.

Document:

- How the discriminator is stored.
- How the MongoDB driver serialises each detail type.
- How unknown or new activity types are handled.

Extension 4: Add Document Validation and Indexes

Add indexes supporting the use cases, such as an index on:

```
studentNumber + activityDate
```

Consider document validation for shared required fields and selected activity-specific fields.

Extension 5: Update the Implementation Class Diagram

Add the Research Activity Log module to the Week 5 implementation diagram. Include:

- Controller
- Service
- Repository
- Document classes
- MongoDB client or database dependency
- Students-module lookup contract, if used
- Razor views

Keep the relational database and document database as separate persistence technologies.

Extension 6: Reverse-Engineer the Document Model

Inspect the stored MongoDB documents and create a document-schema diagram representing the implemented collection.

Compare the reverse-engineered model with:

- The supplied domain model.
- The example JSON documents.
- The C# document classes.

- The relational modelling attempt.

Record which business meaning can and cannot be recovered from the stored documents alone.

Extension 7: Compare SQL and NoSQL Approaches

Complete a structured comparison using:

- Relationship representation
- Schema flexibility
- Nullability
- Duplication
- Query patterns
- Transactions
- Referential integrity
- Adding new activity types
- Reporting
- Schema evolution

Do not conclude that one technology is always better. Explain which model better supports a particular capability and access pattern.

Reflection Questions

1. Why are activity-specific details embedded in the activity document?
2. Why is the complete Student not embedded?
3. What would be different in a normalised relational model?
4. What responsibility belongs to the repository?
5. How did the required access pattern influence the document structure?

Keeping the Project and Models

Keep:

- The completed relational implementation from Lab 1.
- The Research Activity domain model.
- The relational modelling attempt.
- The document model and sample JSON.
- The document-persistence implementation.
- Seed data.
- Tests and design notes.
- Updated diagrams and comparisons, if completed.

Save or commit the working project:

```
git status
git add .
```

```
git commit -m "Complete Week 5 document data architecture"
```